Journal of Computational Science 99 (2026) 102933

Contents lists available at ScienceDirect

Journal of Computational Science

journal homepage: www.elsevier.com/locate/jocs

Low-latency hybrid CORDIC framework on FPGAs☆

Pawel Poczekajlo [a], Ewa Deelman [b], Michela Taufer [c], Pawel Gepner [d,*], Leonid Moroz [d], Jerzy Krawiec [d], Andrzej Przybyl [e]

[a] Koszalin University of Technology, Faculty of Electronics and Computer Science, Koszalin, 75-453, Poland
[b] University of Southern California, USC Information Sciences Institute, Marina del Rey, 90292, CA, USA
[c] University of Tennessee, Department of Electrical Engineering and Computer and Science, Knoxville, 37996, TN, USA
[d] Warsaw University of Technology, Faculty of Mechanical and Industrial Engineering, Warszawa, 02-524, Poland
[e] Czestochowa University of Technology, Department of Artificial Intelligence, Częstochowa, 42-201, Poland

ARTICLE INFO

Keywords:
CORDIC
Algorithm
Recoded
FPGA
Latency

ABSTRACT

CORDIC-based architectures remain a key building block for implementing elementary transcendental functions on resource-constrained FPGAs. However, classical CORDIC suffers from long latencies, irregular iteration patterns in hyperbolic mode, and non-trivial scale-factor compensation. This work builds on a low-latency recoding CORDIC for trigonometric functions based on look up tables anchoring, angle recoding and linear interpolation. It extends that framework to support the hyperbolic mode and the exponential function $\exp(x)$, and validates it across two FPGA families (Intel/Altera Cyclone IV and AMD/Xilinx Spartan-7). The proposed recoded hyperbolic kernel uses a monotonic shift schedule, eliminates runtime direction computation, and leverages a bit-accurate fixed-point model to derive all constants and interpolation coefficients. Experimental evaluation with 16-bit fixed-point operands compares the proposed design with vendor CORDIC and exp IP cores. For $\exp(x)$, the framework achieves a latency reduction of about 36% on the Spartan-7 device relative to the AMD IP core, while using a similar number of LUTs; on Cyclone IV it reduces logic consumption by around 10% and nearly halves the latency with respect to the Intel exp IP core. For sine and cosine, the recoded architecture requires substantially fewer LUTs than vendor CORDIC blocks while preserving numerical accuracy. Overall, the extended framework provides a unified, portable and low-latency solution for trigonometric and exponential computations on FPGAs, suitable for edge and embedded environments.

1. Introduction

The main drawback of the classical CORDIC method [1–3] is its low speed due to its linear convergence (only one correct bit of the result per iteration) and the hardware complexity associated with the need to implement three simultaneous iteration equations for $x_{i+1}, y_{i+1}, z_{i+1}$ when applying a pipeline structure to the computation:

$$\begin{aligned} x_{i+1} &= x_i - \sigma_i 2^{-i} y_i; \\ y_{i+1} &= y_i + \sigma_i 2^{-i} x_i; \\ z_{i+1} &= z_i - \sigma_i \arctan(2^{-i}); \\ \sigma_i &= \text{sign}(z_i),\ i = (0, \ldots, m) \end{aligned} \tag{1}$$

that needs $m+1$ elementary rotations.

To reduce the number of iterations, hybrid structures have been developed that use three steps sequentially: look up tables (LUT) + CORDIC + piecewise-linear multiplication (linear approximation) [4–6]. To simplify the hardware complexity, a CORDIC method with angle recoding has been proposed [7–12], which reduces the system CORDIC iterations (1) to only two equations for x_{i+1}, y_{i+1}. The simplest in terms of hardware implementation is the CORDIC angle recoding method [7,13]. Recently, there have been several new publications on the topic of angle transcoding in CORDIC, one of them being [12]. The theory described therein does not allow for flexible separation of iterations between memory table size and CORDIC. Therefore, it cannot lead to a significant reduction in latency and an increase in the operating frequency of the CORDIC. The algorithm in [7,13] exhibits a drawback as large size of memory (type LUT) is required for large values of m (a table of size not less than $2^{(m/3)} \times m$ bits is needed without the possibility of its reduction). Additionally, the output multipliers are implemented in the {-1,1} basis, preventing the use of multipliers that are part of the DSP blocks in modern FPGA devices.

☆ This article is part of a Special issue entitled: 'AI, HPC &Simulations' published in Journal of Computational Science.

* Corresponding author.
E-mail address: pawel.gepner@pw.edu.pl (P. Gepner).

https://doi.org/10.1016/j.jocs.2026.102933
Received 10 January 2026; Received in revised form 29 April 2026; Accepted 8 June 2026
Available online 12 June 2026
1877-7503/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

These limitations are even more evident in the hyperbolic mode of CORDIC, which is required for functions such as $\exp(x)$ and features repeated iteration indices and a more complex scale compensation.

In our earlier work [14], we proposed a new approach to angle recoding that allows flexible adjustment of the memory table size and the number of CORDIC iterations. This approach eliminated the runtime evaluation of the rotation direction, reduced the number of required CORDIC iterations and enabled significant improvements in latency, logic utilization and numerical accuracy for sine and cosine computations implemented on FPGA.

This work substantially extends, both in scope and in technical depth and makes the following contributions. First, we generalize the proposed LUT–CORDIC–interpolation framework to the hyperbolic mode, enabling efficient computation of the exponential function. We introduce new hardware architecture: a recoded hyperbolic CORDIC scheme with LUT vector initialization for $\exp(x)$. This design inherits the structural advantages of the original method while addressing specific challenges of the hyperbolic iteration sequence, such as repeated microrotation indices and the need for accurate handling of the hyperbolic scaling factor.

Second, the fixed-point modeling approach introduced in [14] is substantially extended and generalized. The bit-accurate software model, which previously covered only the trigonometric mode, now includes the complete datapaths for hyperbolic function $\exp(x)$, including normalization, LUT anchoring, recoded hyperbolic microrotations and linear interpolation. This extended model is used to derive the constant shift schedules, the precomputed *atanh* values and the interpolation coefficients used in the RTL implementation of the new architectures

Third, in addition to the original implementation on Intel/Altera Cyclone FPGA, we include new implementations and benchmarks on the AMD/Xilinx FPGA platform. This allows us to evaluate portability, architecture scalability, resource utilization and maximum operating frequency across two different vendor tool chains and FPGA fabrics. We present comprehensive comparisons with classical hyperbolic CORDIC as well as with IP-core based solutions, highlighting the latency, throughput and resource advantages of the proposed designs.

The combination of LUT-based anchoring, angle recoding and linear interpolation provides a flexible design space where accuracy, latency, memory size and resource usage can be independently tuned. The proposed extensions show that this approach is not limited to trigonometric functions but forms a unified framework for efficient hardware computation of a broad class of elementary functions. The framework thus establishes a general methodology for the low-latency and resource-efficient implementation of hyperbolic functions on FPGA architectures.

2. Description of the proposed method

2.1. Known approaches to angle recoding

Let us consider an arbitrary angle θ represented in a binary fractional format

$$\theta = \sum_{i=0}^{m} a_i 2^{-i}, \tag{2}$$

where $a_i \in \{0, 1\}$. In classical CORDIC, this angle is instead decomposed using a signed basis

$$\theta = \sum_{i=0}^{m} \sigma_i \arctan(2^{-i}), \tag{3}$$

where $\sigma_i \in \{-1, 1\}$. The unit vector $(x_0, y_0) = (1, 0)$ is then iteratively rotated [4].

One well-known drawback of classical CORDIC is its fixed sequential iteration flow and the need to maintain the scaling factor

$$P = \prod_{i=0}^{m} (1 + 2^{-2i})^{-1/2}, \tag{4}$$

which prevents skipping iterations. A second drawback is the runtime evaluation of the next rotation direction σ_i, which depends on intermediate values of z_i.

To eliminate the second drawback, angle recoding replaces the binary basis with a signed basis $b_i = 2a_i - 1$, which directly determines the rotation direction without evaluating σ_i. This technique is commonly known as angle recoding [7–10,12]. Different recoding conventions appear in the literature, differing in the scaling of the basis:

$$\theta_r = \sum_{i=0}^{m} b_i 2^{-i}, \tag{5}$$

$$\theta_r = \sum_{i=0}^{m} b_i 2^{-i-1}, \tag{6}$$

etc. In this work we use (6), giving

$$\theta_r = \sum_{i=0}^{m} (2a_i - 1) 2^{-i-1} = -(1/2)^{m+1} + \theta, \tag{7}$$

so that

$$\theta = \theta_r - \theta_c, \qquad \theta_c = -(1/2)^{m+1}. \tag{8}$$

2.2. Our approach to recoding CORDIC

If the recoded angle is embedded into the arctangent system, we obtain

$$\theta_{rr} = \sum_{i=0}^{m} b_i \arctan(2^{-i-1}), \tag{9}$$

and hence

$$\theta = \theta_{rr} - \theta_{rc}, \qquad \theta_{rc} = -\sum_{i=0}^{m} \arctan(2^{-i-1}). \tag{10}$$

Using the classical CORDIC iteration with signed coefficients b_i, we obtain

$$x_{i+1} = x_i - b_i y_i 2^{-i-1}, \qquad y_{i+1} = y_i + b_i x_i 2^{-i-1}, \tag{11}$$

with the appropriately adjusted initial vector

$$x_0 = P\cos(-\theta_{rc}), \qquad y_0 = P\sin(-\theta_{rc}). \tag{12}$$

In this work, we retain the recoding principle but generalize the structure of the computation using a three-component decomposition:

$$\theta = \theta_1 + \theta_2 + \theta_3,$$

where θ_1 is processed by a small LUT, θ_2 by the recoded CORDIC kernel, and θ_3 by a linear output correction. This decomposition forms the basis for the unified LUT–CORDIC–interpolation framework extended in this journal version to hyperbolic functions.

We define

$$\theta_1 = \sum_{i=0}^{m_1} a_i 2^{-i}, \qquad \theta_2 = \sum_{i=m_1+1}^{m_2} a_i 2^{-i}, \qquad \theta_3 = \sum_{i=m_2+1}^{m} a_i 2^{-i}. \tag{13}$$

The LUT stores precomputed values

$$x_{m_1} = P\cos(\theta_1 - \theta_{rc}), \qquad y_{m_1} = P\sin(\theta_1 - \theta_{rc}), \tag{14}$$

while θ_2 is evaluated using recoded CORDIC iterations

$$x_{i+1} = x_i - b_i y_i 2^{-i-1}, \qquad y_{i+1} = y_i + b_i x_i 2^{-i-1}. \tag{15}$$

Finally, the residual component θ_3 and the lag angle correction are merged into a single linear post-rotation

$$x' = x - zy, \qquad y' = y + zx,$$

which in the extended framework will also serve as the linear correction stage for the hyperbolic kernels computing $\exp(x)$.

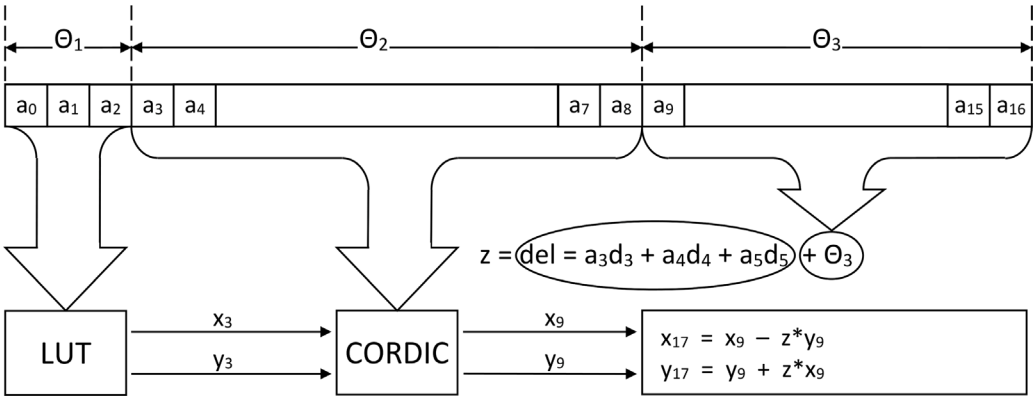


Fig. 1. Schematic view of the modified, recoded CORDIC algorithm.

Algorithm 1 Pseudocode of the proposed algorithm

Input a in binary format
$a \leftarrow \{0,1,1,1,1,0,1,1,1,1,0,0,1,0,0,1,1\}$
$xx \leftarrow \{1037958, 973957, 849399, 672030,$
$452877, 205567, -54525, -311226\}$
$yy \leftarrow \{128268, 381076, 610190, 801365, 942715,$
$1025452, 1044432, 998473\}$
$x \leftarrow \{0,0,0,0,0,0,0,0,0,0\}$
$y \leftarrow \{0,0,0,0,0,0,0,0,0,0\}$
$j \leftarrow a[0] \times 4 + a[1] \times 2 + a[2]$
$x[3] \leftarrow xx[j]$
$y[3] \leftarrow yy[j]$
for $i \leftarrow 3$ **to** 8 **do**
  if $a[i] == 1$ **then**
    $x[i+1] \leftarrow x[i] - (y[i] >> (i+1))$
    $y[i+1] \leftarrow y[i] + (x[i] >> (i+1))$
  else
    $x[i+1] \leftarrow x[i] + (y[i] >> (i+1))$
    $y[i+1] \leftarrow y[i] - (x[i] >> (i+1))$
  end if
end for
$d3 \leftarrow 170$
$d4 \leftarrow 21$
$d5 \leftarrow 3$
$del \leftarrow a[3] \times d3 + a[4] \times d4 + a[5] \times d5$
$Theta3 \leftarrow a[9] \times 2048 + a[10] \times 1024 + a[11] \times 512 + a[12] \times 256 + a[13] \times 128 +$
$a[14] \times 64 + a[15] \times 32 + a[16] \times 16$
$z \leftarrow del + Theta3$
$x17 \leftarrow x[9] - ((z \times y[9]) >> 20)$
$y17 \leftarrow y[9] + ((z \times x[9]) >> 20)$
return $x17, y17$

2.3. Uniqueness of the approach

The key feature of the method is the decomposition of an angle (or argument) into a LUT-based anchor, a recoded CORDIC core, and a final linear correction. Our modified, recoded CORDIC algorithm is presented in the schematic view in Fig. 1 and presented scheme is directly linked to the pseudo-code of the Algorithm 1.

The constants used in Algorithm 1 are fixed-point encoded residual microrotation contributions determined from the bit-accurate software model. In particular, $d_3 = 170$, $d_4 = 21$, and $d_5 = 3$ correspond to the weighted contributions of bits a_3, a_4, and a_5, respectively, after LUT anchoring. Likewise, the values $2048, 1024, \ldots, 16$ are the Q20 positional weights used to reconstruct the residual angle term from bits a_9 through a_{16}. The arrays xx and yy store precomputed anchor vectors associated with the eight possible 3-bit prefixes a_0, a_1, a_2 .

2.4. Hyperbolic CORDIC background and limitations

The hyperbolic CORDIC algorithm evaluates functions such as $\sinh(x)$, $\cosh(x)$ and $\exp(x)$ using the iterative scheme

$$x_{i+1} = x_i + d_i y_i 2^{-i}, \qquad y_{i+1} = y_i + d_i x_i 2^{-i}, \tag{16}$$

where $d_i \in \{-1, 1\}$ is the rotation direction. The angle accumulation uses

$$z_{i+1} = z_i - d_i \operatorname{atanh}(2^{-i}). \tag{17}$$

Hyperbolic CORDIC is known to suffer from several limitations [15, 16]:

- **Non-monotonic iteration schedule:** unlike the circular case, the hyperbolic CORDIC iteration set is not strictly increasing. Importantly, the difficulty is not that the product $\prod_i (1 - 2^{-2i})$ itself diverges; rather, the classical hyperbolic CORDIC requires repeated iteration indices in order to preserve the validity of the hyperbolic microrotation decomposition and to ensure convergence of the iterative trajectory over the desired argument range. Consequently, certain indices must be repeated (e.g., $i = 4, 13, 40, \ldots$), resulting in a non-monotonic and irregular iteration schedule [15,16]. This irregularity complicates pipelined hardware implementations.
- **Large scaling constant:** the hyperbolic gain $K_h = \prod_i \sqrt{1 - 2^{-2i}}^{-1}$ with repeated i, grows quickly with the iteration count and must be explicitly compensated in hardware. Unlike the circular case, no simple closed-form expression exists. This limitation is well documented [15].
- **Ambiguity of rotation direction:** in the hyperbolic mode the direction term $d_i = \operatorname{sign}(z_i)$ must be evaluated at every iteration. This makes the algorithm inherently sequential and data-dependent, since each step depends on the updated residual angle z_i. As noted in [15], this runtime decision significantly limits pipelining efficiency and increases latency in FPGA realizations.

These limitations motivate us to adopt a fully recoded hyperbolic CORDIC kernel for $\exp(x)$, in which the argument is directly embedded in a control word. By combining argument anchoring, recoded microrotations and a small post-rotation, we eliminate runtime sign evaluation, avoid explicit index repetition, and keep a fixed, small number of CORDIC iterations.

2.5. Hyperbolic CORDIC-based computation of exp(x)

The exponential function is evaluated utilizing the property that $\exp(x) = \sinh(x) + \cosh(x)$. Since the hyperbolic CORDIC algorithm intrinsically tracks the vector components (X, Y) corresponding to $(\cosh z, \sinh z)$, it provides a natural framework for this computation.

While the classical algorithm requires irregular index repetitions and runtime direction decisions, our proposed framework employs a *fully recoded* approach. The input argument x is mapped to a finite bit vector $a = (a_0, \dots, a_{16})$, which simultaneously encodes the LUT anchor, the sequence of microrotations, and the residual correction. This results in a deterministic datapath with fixed latency.

2.5.1. Argument encoding and input range decomposition

The input argument x is decomposed into three components:

$$x = \theta_1 + \theta_2 + \theta_3. \tag{18}$$

where θ_1 is the coarse component, θ_2 covers the medium-range microrotations and θ_3 represents the fine residual angle.

The coarse component θ_1 is determined by the three most significant bits (a_0, a_1, a_2), which form the LUT index

$$j = 4a_0 + 2a_1 + a_2, \qquad j = 0, \dots, 7.$$

For each index j, the LUT stores a precomputed anchor vector (X_3, Y_3).

The LUT entries are not heuristic constants. Each pair (X_3, Y_3) represents a scaled hyperbolic vector corresponding to a coarse rotation angle $\theta_{1,j}$. The anchor vectors are obtained offline from the analytical model

$$(X_3, Y_3) \approx P_{\text{scale}}(\cosh\theta_{1,j}, \sinh\theta_{1,j}),$$

where the parameters $(P_{\text{scale}}, \theta_0, \Delta\theta)$ defining the angle grid $\theta_{1,j} = \theta_0 + j\Delta\theta$ are estimated by least–squares fitting to a bit-accurate fixed-point reference implementation of the complete CORDIC pipeline. This calibration implicitly accounts for the cumulative scaling of the subsequent microrotations and the linear correction stage. The final values are quantized into the Q20 format and stored in the LUT as the arrays `xx_h` and `yy_h`.

In this way, the LUT replaces the coarse hyperbolic rotations while preserving numerical accuracy; the remaining global scaling is compensated by a single constant factor $P \approx 0.2995$ applied at the output.

The medium-scale component θ_2 is handled by a sequence of recoded microrotations controlled by bits $a_3, \dots, a_8$.

Finally, the fine component θ_3 represents the residual angle. To avoid runtime evaluation of transcendental functions, θ_3 is constructed as a linear combination of the input bits, corrected by a compensation term δ:

$$\theta_3 = \frac{z}{2^{20}}, \qquad z = \delta(a_3, a_4, a_5) + \sum_{k=0}^{7} a_{9+k} \cdot 2^{11-k}. \tag{19}$$

The term $\delta(\cdot)$ compensates for the discretization error introduced by approximating the hyperbolic rotation angles $\operatorname{atanh}(2^{-k})$ with their linear terms 2^{-k} for the indices $k \in \{4, 5, 6\}$. This correction allows the residual angle to be reconstructed using simple integer arithmetic (shift-and-add) rather than complex function evaluations, matching the weights used in the hardware model $(170, 21, 3)$.

2.5.2. Recoded hyperbolic microrotations

The microrotation stage is fully deterministic. The direction of rotation d_i is derived directly from the input bits:

$$d_i = 2a_i - 1, \qquad \text{for } i = 3, \dots, 8. \tag{20}$$

Consequently, $d_i = +1$ if $a_i = 1$, and $d_i = -1$ otherwise. Unlike the standard hyperbolic CORDIC, which requires direction evaluation based on the intermediate variable z_i, our coefficients are known *a priori*.

Starting from the LUT anchor (X_3, Y_3), the kernel performs six iterations. The shift sequence is adapted to the LUT start point, using a monotonic schedule $S_i = i + 1$:

$$\begin{aligned} X_{i+1} &= X_i + d_i\, Y_i\, 2^{-(i+1)}, \\ Y_{i+1} &= Y_i + d_i\, X_i\, 2^{-(i+1)}. \end{aligned} \tag{21}$$

This strictly increasing shift sequence $(4, 5, \dots, 9)$ avoids the need for index repetitions (e.g., $i = 4, 13$) required in standard hyperbolic CORDIC to ensure convergence. This regularity significantly simplifies the hardware control logic.

2.5.3. Linear post-correction and reconstruction

After the microrotations, the vector (X_9, Y_9) is rotated by the residual angle θ_3. Since θ_3 is small, we approximate the hyperbolic rotation using the first-order Taylor expansion ($\cosh\theta_3 \approx 1$, $\sinh\theta_3 \approx \theta_3$):

$$\begin{aligned} X_{17} &= X_9 + z \cdot Y_9 \cdot 2^{-20}, \\ Y_{17} &= Y_9 + z \cdot X_9 \cdot 2^{-20}. \end{aligned} \tag{22}$$

This step corresponds to the final linear correction in the reference code, effectively absorbing the remaining approximation error.

The final value of the exponential function is reconstructed by summing the hyperbolic components and applying a global normalization:

$$\exp(x) \approx (X_{17} + Y_{17}) \cdot K_{\text{norm}}. \tag{23}$$

In a classical hyperbolic CORDIC with a full sequence of 17 microrotations the overall gain is given by

$$K_{\text{gain}} = \prod_{i=1}^{17} \sqrt{1 - 2^{-2i}}. \tag{24}$$

This expression, however, assumes that all rotations are performed explicitly and that the rotation directions are determined at run time.

In the proposed architecture, a different situation occurs: the first three microrotations are absorbed into the LUT anchor points (X_3, Y_3), the next six are executed in the recoded CORDIC kernel, and the remaining least–significant rotations are replaced by the linear post-correction stage. As a consequence, the effective gain of the system cannot be computed directly from the classical product above (24).

Consequently, the global normalization constant $K_{\text{norm}} = \frac{1}{K_{\text{gain}}}$, is determined numerically using a bit-accurate fixed-point reference model that mirrors the RTL datapath. For the adopted word length and iteration schedule we obtain

$$K_{\text{norm}} \approx 0.2995,$$

and this value is implemented in hardware as a single multiplication by a fixed constant. This calibration compensates simultaneously for the CORDIC gain, LUT pre-scaling and linear post-rotation, ensuring that the reconstructed value matches the natural scale of $\exp(x)$.

The overall fixed-point computation of the recoded hyperbolic CORDIC kernel for $\exp(x)$ is summarized in Algorithm 2.

Similarly to the trigonometric case, the constants used in Algorithm 2 are fixed-point encodings derived from the bit-accurate hyperbolic model. The values $d_3 = 170$, $d_4 = 21$, and $d_5 = 3$ represent residual contributions of the corresponding recoded bits after LUT anchoring, while the array `weight[0..7]` = {2048,1024,512,256,128,64,32,16} defines the Q20 positional weights used to reconstruct the residual argument from bits a_9 through a_{16}. The arrays `xx_h` and `yy_h` contain precomputed hyperbolic anchor vectors for the eight possible 3-bit prefixes $a_0a_1a_2$.

3. FPGA implementation and results

The hardware implementation is done on two platforms: Intel (Altera) and AMD (Xilinx). For Intel platform, realization is in Verilog language in the Quartus Prime environment for the Cyclone EP4CE115 FPGA chip. For AMD platform, realization is in VHDL language in the AMD Vivado Design Suite environment for the Spartan-7 XC7S50CSGA324-1 chip. The Intel Cyclone IV chip is manufactured using 60 nm technology. The second reprogrammable AMD Spartan-7 chip is manufactured using 28 nm technology. These represent two different generations of chips. This hardware selection allows for verification of correct operation on technologically different FPGAs.

Algorithm 2 Recoded hyperbolic CORDIC kernel for $\exp(x)$.

Input: bit vector $a[0..16]$
$\texttt{xx_h}[317215, 336983, 377820, 442696, 535212, 661350, 829041, 1048568]$,
$\texttt{yy_h}[38496, 120059, 208818, 311186, 432792, 581585, 766920, 1000176]$
$d_3 \leftarrow 170,\ d_4 \leftarrow 21,\ d_5 \leftarrow 3$
$\texttt{weight}[0..7] \leftarrow \{2048, 1024, 512, 256, 128, 64, 32, 16\}$
$K_{\text{norm}} \approx 0.2995$ {global scaling factor}
$x[0..9] \leftarrow 0$
$y[0..9] \leftarrow 0$
$j \leftarrow 4a_0 + 2a_1 + a_2$
$x[3] \leftarrow \texttt{xx_h}[j]$
$y[3] \leftarrow \texttt{yy_h}[j]$
for $i \leftarrow 3$ **to** 8 **do**
  if $a[i] = 1$ **then**
    $x[i+1] \leftarrow x[i] + (y[i] >> (i+1))$
    $y[i+1] \leftarrow y[i] + (x[i] >> (i+1))$
  else
    $x[i+1] \leftarrow x[i] - (y[i] >> (i+1))$
    $y[i+1] \leftarrow y[i] - (x[i] >> (i+1))$
  end if
end for
$\delta \leftarrow a[3] \cdot d_3 + a[4] \cdot d_4 + a[5] \cdot d_5$
$\theta_3 \leftarrow 0$
for $k \leftarrow 0$ **to** 7 **do**
  $\theta_3 \leftarrow \theta_3 + a[9+k] \cdot \texttt{weight}[k]$
end for
$z \leftarrow \delta + \theta_3$ {z in Q20 format}
$x17 \leftarrow x[9] + ((z \cdot y[9]) >> 20)$
$y17 \leftarrow y[9] + ((z \cdot x[9]) >> 20)$
$\exp_approx \leftarrow \dfrac{x17 + y17}{2^{20} \cdot K_{\text{norm}}}$
return $\exp_approx$

All implementations of algorithms in the FPGA chips are made of basic logic elements (e.g. gates, registers, flip-flops) — without DSP blocks (to have a fair comparison with other implementations). The DSP-free approach is also more versatile. Implementation is possible on any FPGA (many simple reprogrammable devices do not have DSP blocks). Selected mathematical functions are implemented using the presented algorithms. At the same time, in order to compare the results, the same functions are implemented using ready-made (built-in) libraries. The implemented algorithms operate on signed values in many places. For compatibility with libraries and standard processing tools, U2 encoding is used throughout — the most significant bit (MSB) has a negative weight, the remaining bits have a positive weight. This encoding is characterized by an asymmetric range, which in many places requires consideration of the lack of full coverage for negative and positive numbers. In most cases, code optimization is by default and follows the standard configuration of the development environment. This approach ensures reliable comparisons between implementations.

3.1. Implementation of CORDIC-based sin() and cos() functions

Fig. 2 shows a block diagram of the proposed algorithm for sin() and cos() functions in an FPGA. Fig. 4 presents the function block diagram of our version (from Quartus Prime environment). The proprietary algorithm works as follows. The LUT returns elements according to the position number provided at the input. Subsequent iterations are implemented as a single module, called repeatedly. Shifts within the modules must be arithmetic (the sign of the value must be preserved). However, the shift itself can be performed by connecting lines, shifting the bits in the appropriate direction and doubling the line of the highest-order bit. At the outputs of the iteration blocks, the number of bits is matched to the corresponding inputs of the subsequent blocks. Otherwise, the number of bits in subsequent blocks would multiply, and the final blocks would operate on several hundred bits. Since the final result is reduced to a predetermined number of bits, this would be a significant redundancy and an unnecessary increase in the hardware system's occupancy. The called adders are implemented as universal "add/sub" blocks controlled by an operation control bit.

Using standard Quartus environment libraries (Altera IP Core), the Altera_CORDIC function was also analyzed. For the Vivado environment, Xilinx IP Core libraries are used, from which Xilinx_CORDIC has been implemented. Both functions provides the ability to calculate the values of trigonometric functions sin() and cos(). The libraries (via a built-in configuration tool) offers extensive possibilities for setting the key parameters of the implemented Altera_CORDIC and Xilinx_CORDIC functions. Identical outputs precision was used, retaining 16 bits of the fractional part in fixed-point notation. The number of precision bits is also configured for inputs. Only the number of fractional bits can be changed. The integer bits are imposed by the range of output values. The output result is adjusted to the specified register length by rounding to the nearest available representation (while retaining the sign of the value). Support for floating-point formats (inputs and outputs) is not available for these functions. The computing system (combinational-iterative) can be configured to operate in time mode (group delay) or step mode (control by a clock signal with a specified minimum frequency). Altera_CORDIC and Xilinx_CORDIC implementations was performed without DSP blocks (only on logic elements). The code describing the computational functions can be generated in HDL languages (VHDL or Verilog can be selected). The available Altera_CORDIC library is very extensive and allows the following mathematical operations to be performed:

- Atan2 Function - calculation of the arctangent function for two (required) arguments entered as input data;
- Vector Translate Function - calculation of the angle (in radians) and length of the vector for the entered point coordinates (relative to the origin of the coordinate system) - arguments as input data;
- SinCos Function - calculation of the trigonometric functions sin() and cos() for a given angle (in radians) entered as input data;

The Xilinx library also allows for the implementation of several different functions (Rectangular <-> Polar Conversion, Trigonometric, Hyperbolic, Square Root).

In line with the subject matter of this article, the implementation of Altera_CORDIC in the SinCos Function (Trigonometric for Xilinx_CORDIC) variants were analyzed. The SinCos Function can be configured in two different ways. The format and range of the input argument (angle) given in radians are specified:

- input angle given without a sign, entered in the range from 0 to 0.5π, the values of the sin() and cos() functions at the output only take the positive range, i.e., from 0 to +1 (in accordance with the angle range, bit redundancy is avoided).
- input angle given with a sign, entered in the range from $-\pi$ to $+\pi$, the values of the sin() and cos() functions at the output take on a positive and negative range, i.e. from −1 to +1 (in accordance with the angle range, full coverage of output values including extreme values — many fixed-position systems with U2 encoding do not return an equal value of +1, which results from the reduced number of integer bits, i.e., only one integer bit with a negative weight of −1, and bits with positive weights are only fractional).

The full configuration of the SinCos Function from the Altera_CORDIC library also includes specifying the exact number of bits with the precision part (fractional bits) defined for both the input and output (available range from 1 to 64 bits), and selecting the data type option: signed (U2 encoding) or unsigned (which obviously limits the range of the function). At the same time, a restriction is imposed: the number of precision bits (fractional) at the function output cannot be greater than the number of precision bits at the function input. The presented proprietary algorithms based on CORDIC do not have this restriction. This should not be interpreted as creating additional information beyond the input resolution; rather, it is a practical interface advantage, since the proposed architecture can provide a wider fixed-point output format to preserve guard bits and simplify integration with downstream hardware blocks. Fig. 3 shows the input/output model of both versions of the CORDIC algorithm for sin() and cos().

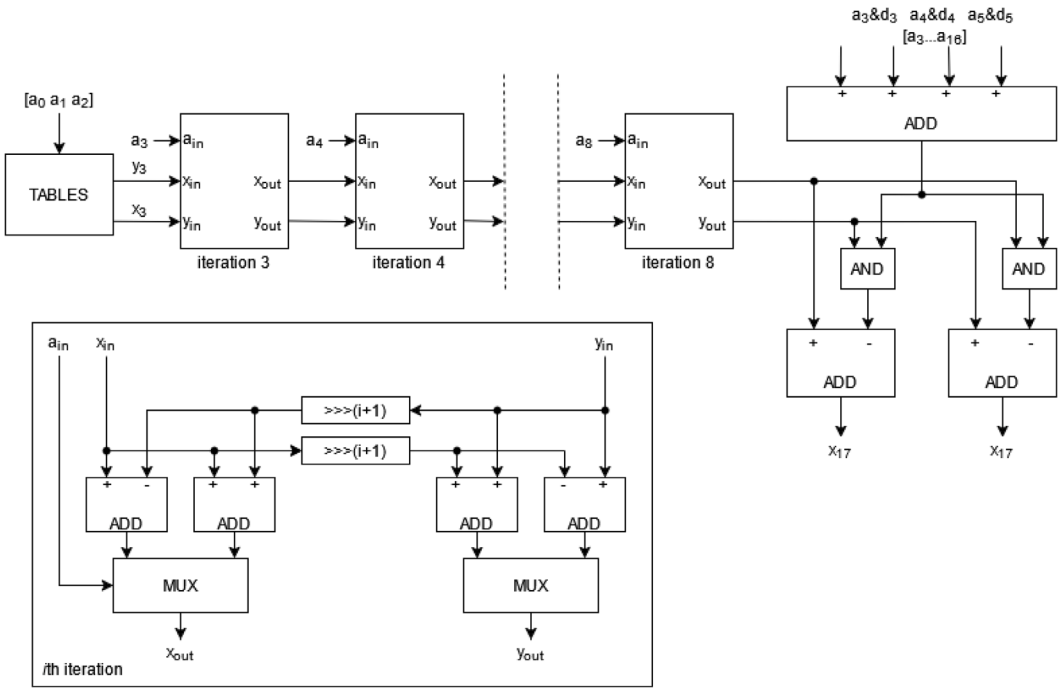


Fig. 2. Block diagram of the implementation of the algorithm for sin() and cos() functions in an FPGA.

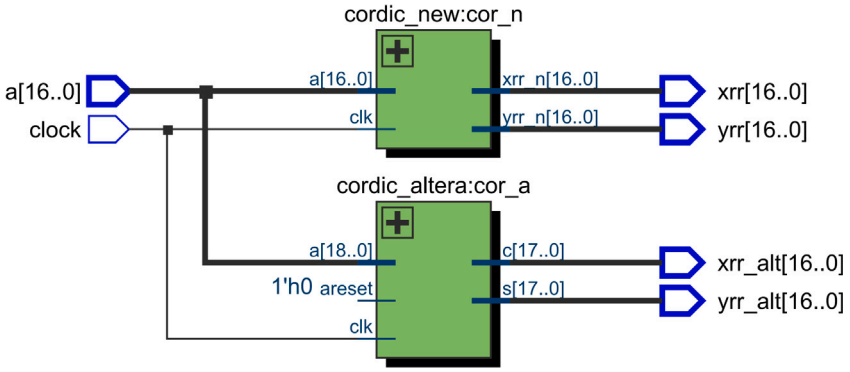


Fig. 3. Implementation model of both versions of the CORDIC sin() and cos() realization algorithm (from RTL tool in Quartus Prime).

3.2. Implementation of CORDIC-based exponential function

The algorithm implementing the exp() function is almost identical to that for the sin() and cos() functions. The change involves only a few operations (as can be seen by comparing the Algorithms 1 and 2). This makes hardware adaptation for the exp() function very simple. Additional summations do not pose an implementation problem, and the resulting increase in occupancy is practically negligible. For hardware implementation (and evaluation of selected parameters), an important change is the multiplication operation (value scaling) performed at the end of the algorithm 2). The division by the factor 2^{20} itself can be performed as a simple projection (connection) of selected bits from a wire or register. On the other hand, division by the constant K_{norm} is implemented as multiplication by the inverse. Since it is a constant, it is easy to optimize the operation, which is performed quickly and consumes few FPGA resources. Fig. 7 presents the function diagram block of the CORDIC-based exponential function (from Quartus Prime environment). Fig. 6 shows the input/output model of the algorithm for exp().

The exp() functions from the Intel Quartus FP_FUNCTION library (FP_FUNCTIONS Intel FPGA IP Core) and AMD library (AMD FPGA IP Core) has also been implemented. Only the version operating on floating point values (single or double precision, selectable) is available. This applies to both the input and output of the function. In order to use identical input data (in fixed-point format with 16 bits of fractional part), two additional format conversion blocks were used (as shown in Fig. 5):

- from fixed point to floating representation,
- from floating point to fixed representation.

The exp() function from the Intel library is available in three variants: base *e*, base 2, and base 10. Only step-by-step (iterative) execution is possible. According to the configurator, the minimum number of iterations that can be declared is 6. The exp() function from the AMD library is more flexible and allows implementation as a combinational circuit without iteration.

3.3. Testing scenario and evaluation methodology

The implementation of algorithms and various functions on FPGA (and CPLD) systems differs significantly from other typical computing platforms (e.g., CPU and GPU). The difference is already apparent at

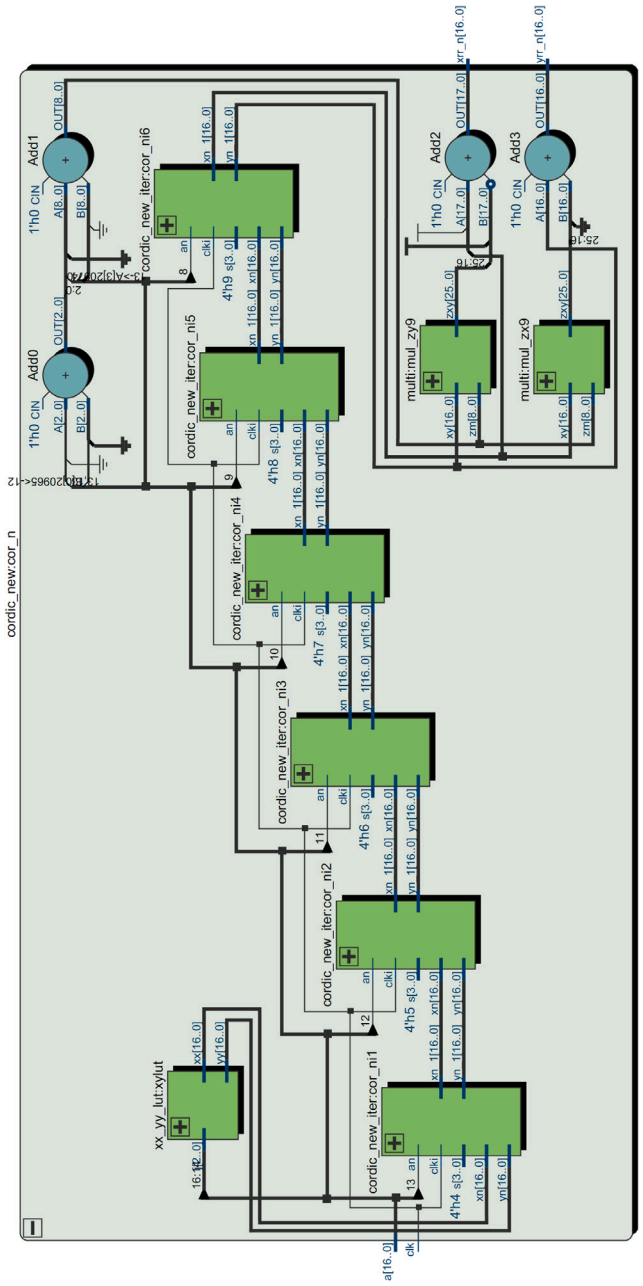


Fig. 4. Implementation block diagram of the proposed CORDIC sin() and cos() realization (from RTL tool in Quartus Prime).

the level of programming languages, where HDL (VHDL/Verilog) hardware description languages are essentially unique to reprogrammable systems (FPGA/CPLD). At the same time, the difference is already at the level of hardware architecture, which forces a completely different approach to application design (code structure) and evaluation methodology.

For typical graphics systems (GPUs) and computer processor architectures (CPUs), the electrical power consumption and energy efficiency are often analyzed, especially for super-efficient computer units. On the other hand, overall performance is usually referred to as the time taken to perform specific tasks with a specified computational accuracy and labor intensity using resources (including bus bandwidth, cache memory, and operating cycles). Many current publications describe and present studies using these metrics [17–21].

Table 1
Results of FPGA occupancy for CORDIC realisation of sin() and cos().

CORDIC Realisation	LUTs
Presented CORDIC algorithm on Intel FPGA chip	1597
Altera_CORDIC with 16-bits fractional outputs	2866
Presented CORDIC algorithm on AMD FPGA chip	909
Xilinx_CORDIC with 16-bits fractional outputs	1202
CORDIC algorithm (as reported [12])	1438
BBR-CORDIC (as reported in [12])	1643
CORDIC II (as reported in [12])	1433

FPGA/CPLD systems are utilized as typical solutions for parallelizing operations (often counted in tens or even hundreds of arithmetic and logical actions). Such capabilities are conducive to the implementation of repetitive calculations performed in an iterative approach (such as the CORDIC algorithm), where many operations are performed in parallel in a pipelined manner.

The metrics typically analyzed and evaluated in FPGA implementations are:

- **Logic Occupation**: This is a key factor limiting the "size" of an algorithm in an FPGA; the number of LUTs, logic gates, flip-flops, or even complex DSP blocks is important for high performance and is directly related to the use of low/high-cost FPGA chips (i.e., cheaper chips have fewer resources).
- **Throughput/Latency**: The delay resulting from the signal path from input to output is most important for real-time processing. By limiting delays (usually to an acceptable level) through parallelization of operations, throughput (system operating frequency) can be increased.
- **Precision/Accuracy**: The implementation specificity of FPGAs allows for precise and almost arbitrary selection of the number of bits (with a resolution of one bit); thanks to this, we can achieve the required (but not excessive) precision and performance with optimal resource and energy consumption.

For testing purposes, 104 input samples (for two algorithms: sin() together with cos() and exp()) are prepared. They are determined with a fit to the bit representation (register lengths) and in selected cases also format conversion. The complete measurement diagram is shown on Fig. 8.

For the purposes of comparing the implemented algorithms, three key metrics were selected:

- FPGA resources used – number of LUTs (Lookup Tables) – value returned by the programming environment in the report after project compilation; results from the number of resources used, such as gates or flip-flops;
- Time delay after which the result is returned (latency) – defined as the minimum processing delay for which the correct result is returned – according to the sampling frequency (for pipeline implementations also expressed in clock cycles).
- Signed error – reported as the maximum positive error and the minimum negative error according to the equations:

$$\begin{aligned} e(j) &= q(j) - q_{fp}(j), \\ e_{\max} &= \max_j e(j), \\ e_{\min} &= \min_j e(j) \end{aligned} \tag{25}$$

for $j = 1, 2, \ldots, 104$, where $q(j)$ is the value obtained for a specific implementation of a given algorithm, and $q_{fp}(j)$ is the reference result for the same input value determined in the SciLab environment [22].

We compare [12] and aggregate the results for sin() and cos() realizations in Tables 1–3.



Fig. 5. Data flow diagram using functions from the FP_FUNCTIONS Intel FPGA IP library.

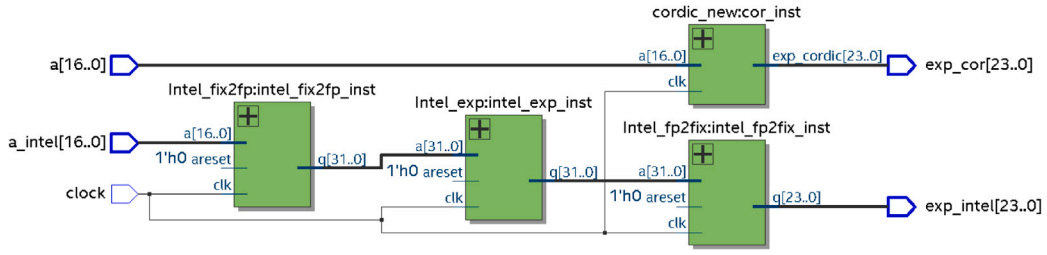


Fig. 6. Implementation model of both versions of the CORDIC exp() realization algorithm (from RTL tool in Quartus Prime).

Table 2
Results of response time (latency) for CORDIC realisation of sin() and cos().

CORDIC Realisation	Time (ns)
Presented CORDIC algorithm on Intel FPGA chip	56
Altera_CORDIC with 16-bits fractional outputs	94
Presented CORDIC algorithm on AMD FPGA chip	24
Xilinx_CORDIC with 16-bits fractional outputs	49
CORDIC algorithm (as reported in [12])	60
BBR-CORDIC (as reported in [12])	60
CORDIC II (as reported in [12])	90

Table 3
Results of maximum positive and minimum negative signed errors for CORDIC realisations of sin() and cos().

CORDIC Realisation	Max signed error	Min signed error
Presented CORDIC algorithm on Intel FPGA chip	4.78e−06 (sin) 4.84e−06 (cos)	−1.12e−05 (sin) −1.27e−05 (cos)
Altera_CORDIC 16-bit fractional outputs	1.33e−05 (sin) 1.37e−05 (cos)	−1.21e−05 (sin) −1.14e−05 (cos)
Presented CORDIC algorithm on AMD FPGA chip	4.78e−06 (sin) 4.84e−06 (cos)	−1.12e−05 (sin) −1.27e−05 (cos)
Xilinx_CORDIC 16-bit fractional outputs	1.81e−05 (sin) 1.68e−05 (cos)	−1.76e−05 (sin) −2.09e−05 (cos)
CORDIC algorithm [12]	3.04e−05 (sin) 3.04e−05 (cos)	−3.04e−05 (sin) −3.04e−05 (cos)

Table 4
Results of FPGA occupancy for exp() functions realizations.

Realization	LUTs
Presented CORDIC realisation of exp() on Intel FPGA chip	1822
exp() function from Intel library (IP Core on Intel FPGA chip)	1998 (+258 for floating ⇔ fixed point conversion)
Presented CORDIC realisation of exp() on AMD FPGA chip	1193
exp() function from AMD library (IP Core on AMD FPGA chip)	1148 (+204 for floating ⇔ fixed point conversion)

Table 5
Results of response time (latency) for exp() functions realizations.

Realization	Time (ns)
Presented CORDIC realisation of exp() on Intel FPGA chip	62 ns
exp() function from Intel library (IP Core on Intel FPGA chip)	33 cycles (latency) + 86 ns
Presented CORDIC realisation of exp() on AMD FPGA chip	34 ns
exp() function from AMD library (IP Core on AMD FPGA chip)	53 ns

Table 6
Results of maximum positive and minimum negative signed errors for exp() function realizations.

Realization	Max signed error	Min signed error
Presented CORDIC realisation of exp() on Intel FPGA chip	2.88929e−06	−9.41167e−05
exp() function from Intel library (IP Core on Intel FPGA chip)	4.05507e−07	−6.88674e−07
Presented CORDIC realisation of exp() on AMD FPGA chip	2.88929e−06	−9.41167e−05
exp() function from AMD library (IP Core on AMD FPGA chip)	7.14636e−07	−5.07769e−07

In addition, it is worth noting that the Altera_CORDIC (sin(), cos() realizations) library used does not get the configured parameters. Declaratively, the implemented version of Altera_CORDIC should return a result in less than 53 ns. The results from Altera_CORDIC for the 104 samples used were unsatisfactory — most of the results were incorrect. As the clocking was lowered, the results improved. The Altera_CORDIC library returned correct results only when feeding samples with a period of 94 ns — much more than the declared value. At the same time, Altera_CORDIC has the highest occupancy, which is due to the poor optimization of the ready-made library 1. The only advantage of this library is the accuracy (precision of calculations), which is comparable with other implementations 3. Presented CORDIC algorithm for sin() and cos() functions has occupancy (LUTs) at a very good level — only slightly higher than the lowest results 1. The advantage of the presented algorithm is the latency time (24 ns) which allows higher sampling (and processing) frequencies 2. In general, both implementations of the proposed algorithm (on AMD and Intel) are faster than other published solutions. The Xilinx_CORDIC library is also very fast here. The differences between the presented algorithm on both FPGAs are quite large (24 ns vs 56 ns). However, this can be easily explained. The AMD Spartan chip is manufactured using 28 nm technology, which, according to the manufacturer's data, allows for up to 30% faster performance than 45 nm generation devices. The Intel Cyclon chip is manufactured using 60 nm technology, so the 57% reduction in execution time is entirely justified. The presented algorithm implemented on the AMD Spartan-7 chip consumes the least

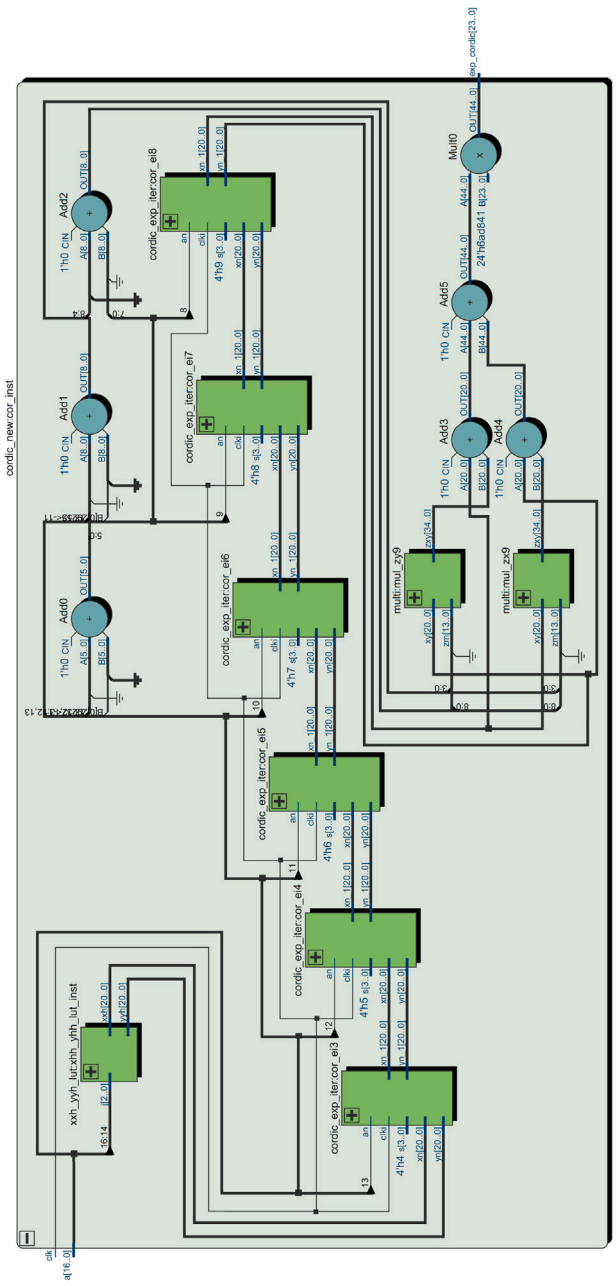


Fig. 7. Implementation block diagram of the proposed CORDIC exp() realization (from RTL tool in Quartus Prime).

resources. Library realization (Xilinx_CORDIC vs Altera_CORDIC) is also less resource-intensive in AMD implementation. This effect may result from the use of the latest Vivado Design Suite environment, which has completely replaced the previous Xilinx ISE. Improvements in the optimization algorithms for implemented operations directly contribute to lower resource utilization.

The aspect of smaller errors is also important (even an order lower than for other algorithms in the case of max errors) 3. Other errors are at similar levels (about 1.2e−05). It is worth noting that the presented CORDIC algorithm gave exactly the same results on Intel and AMD chipsets, which confirms its correct implementation on both platforms. Numerical accuracy is preserved while processing faster. sin() and cos() functions based on the CORDIC algorithm [12] have by far the largest errors, which may be related to almost no hardware occupancy (LUTs). Typically, lower occupancy is achieved by reducing the number of fractional bits, which results in larger errors. At the same time, it is clear that using the latest (optimized) environments and compilers can significantly improve results, as confirmed by implementations on AMD FPGA chips. The Fig. 9 also shows graphs of the output values of the sin() and cos() functions for test samples and the corresponding errors. The observed error pattern is deterministic rather than random. Its piecewise-irregular shape results from the combination of LUT anchoring, discrete recoding boundaries, finite-word-length truncation in the shift-add stages, and the final linear correction. Between anchor transitions the error varies smoothly, while local irregularities appear when the selected LUT segment or recoded bit pattern changes.

In Tables 4, 5 and 6 we compare and aggregate the results for exp() realizations.

The exp(x) algorithm is based on the recoded hyperbolic CORDIC kernel. Thanks to this approach, it was possible to reduce resource usage by 10% compared to the library function exp() from the Intel FPGA IP Core. Both exp() implementations on AMD have comparable resource usage. At the same time, the ready-made libraries operate only on floating-point values, which limits their application. Additional processing blocks for fixed-point values (in both directions) increase the difference in occupancy to almost 20% compared to the presented recoded CORDIC exp() solution. The ready-made library has slightly smaller signed errors ($e_{\max}$ and $e_{\min}$), but this comes at the cost of greater FPGA resource usage (Intel IP Core) or longer total delay (AMD IP Core). At the same time, the proposed algorithm is very flexible and the precision can be easily increased by adding one or two additional bits. The error results obtained for the proposed exp() algorithm remain consistent with the selected 16-bit fractional output precision, where the least significant bit has weight $2^{-16} \approx 1.52588 \times 10^{-5}$. Fig. 10 shows the output values of the exp() function for test samples and the corresponding errors.

The latency results are very interesting. The implementation of the Intel library function does not adhere to the parameters declared in the configurator (similar to Altera_CORDIC). Instead of the declared 6 clock cycles, the latency is as high as 33 cycles (including +2 cycles for fixed-point format support). The inability to run in "one cycle" mode is also a significant limitation. Even the final delay (after the last 33 cycles) is almost 15 ns (≈ 29%) greater than the total group delay (latency) of the proposed algorithm on Intel chip. The timing results show that slightly higher precision (and floating-point implementation) has a very negative impact on latency and the time–frequency parameters of the libraries realizations. As with the sin() and cos() algorithms, here too we can see the positive impact of using a newer generation of FPGA (faster algorithm execution) and a new environment and compiler (lower memory usage).

4. Conclusion

This article presented a framework for the Low-Latency Recoding CORDIC algorithm, which combines angle recoding, LUT-based anchoring, and interpolation to significantly minimize the number of required iterations. This proposed approach effectively eliminates the critical path delay associated with runtime direction evaluation found in conventional CORDIC designs. This work generalizes the methodology for trigonometric functions to the hyperbolic functions, enabling efficient hardware implementation of the exponential function, exp(x).

We successfully adapted the framework to handle the specific challenges of hyperbolic CORDIC, such as repeated microrotation indices and complex scale factor compensation, demonstrating that the solution is versatile for a broader class of elementary functions.

Experimental results confirm that the proposed method offers a superior trade-off between latency, throughput, and hardware complexity compared to classical non-recoded architectures and standard IP cores. Specifically, the quantitative analysis on AMD/Xilinx platforms demonstrates that the proposed hyperbolic architecture achieves a latency reduction of approximately 36% compared to vendor-optimized

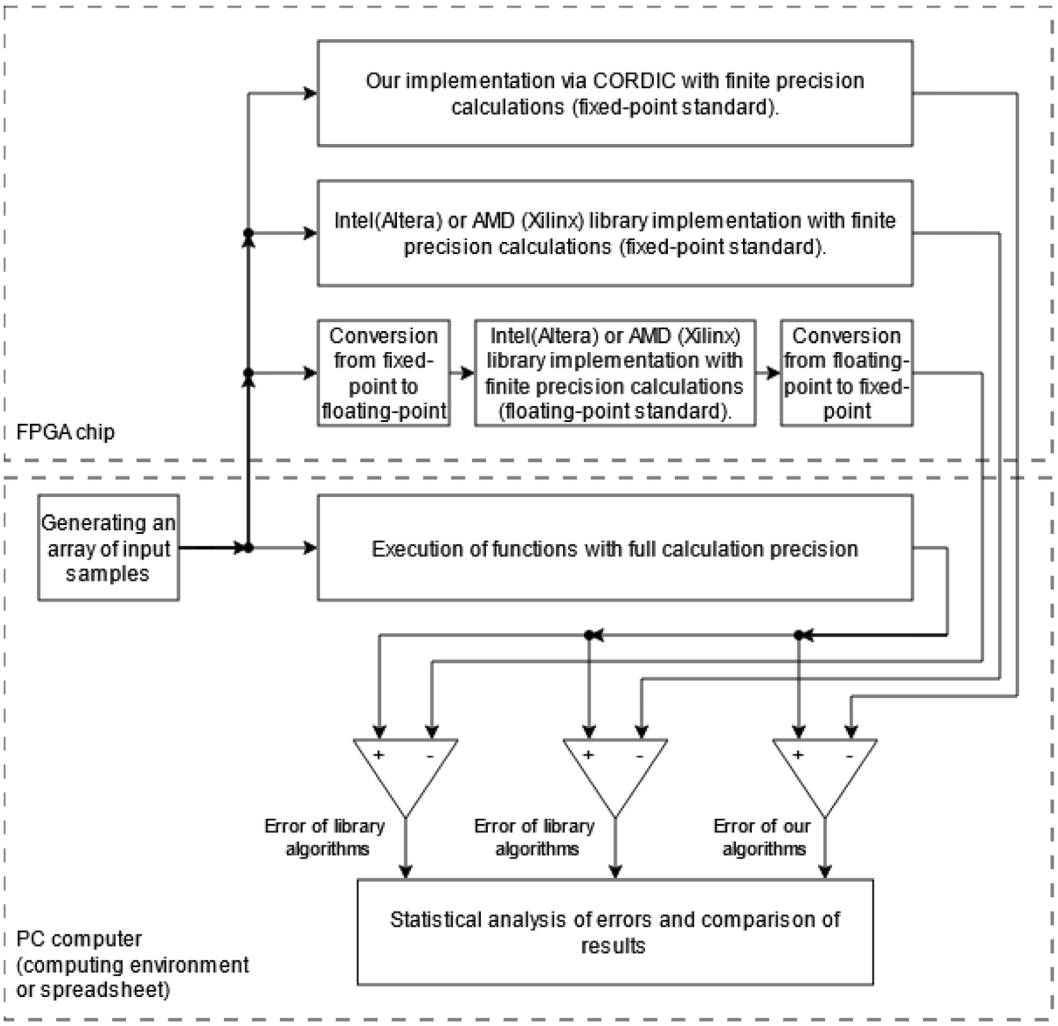


Fig. 8. Measurement scheme of the analyzed implementations of computational algorithms.

IP cores, while maintaining comparable resource utilization. For the Intel/Altera family, the logic consumption was reduced by 10% in the high-speed configuration but latency is almost two times smaller than Intel IP core implementation Unlike previous studies restricted to a single vendor, this work verified the architecture's portability and scalability by benchmarking across these two major FPGA families, confirming that the benefits are consistent across different FPGA fabrics.

Furthermore, the development of a comprehensive bit-accurate software model, now covering hyperbolic datapaths, allows for precise derivation of shift schedules and interpolation coefficients, bridging the gap between theoretical algorithm design and practical RTL implementation.

Future work will focus on further automating the design space exploration to dynamically generate optimal configurations for specific accuracy and area constraints. Another promising direction is a dedicated complex-exponential unit. While a straightforward implementation follows directly from Euler's formula by composing the proposed kernels independently, a jointly designed architecture could exploit shared control and normalization structures to achieve gains in latency and area that independent composition cannot offer.

CRediT authorship contribution statement

Pawel Poczekajlo: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis. **Ewa Deelman:** Writing – review & editing, Writing – original draft, Supervision, Investigation, Formal analysis, Conceptualization. **Michela Taufer:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Conceptualization. **Pawel Gepner:** Writing – original draft, Supervision, Software, Project administration, Methodology, Investigation, Conceptualization. **Leonid Moroz:** Writing – review & editing, Writing – original draft, Investigation, Formal analysis, Conceptualization. **Jerzy Krawiec:** Writing – review & editing, Writing – original draft, Validation, Software, Project administration, Conceptualization. **Andrzej Przybyl:** Writing – review & editing, Validation, Software.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was partly supported by PLGrid Infrastructure at ACK Cyfronet AGH, Krakow, Poland. This work was also partly supported by the National Science Foundation, United States under grant #2331153.

Data availability

No data was used for the research described in the article.

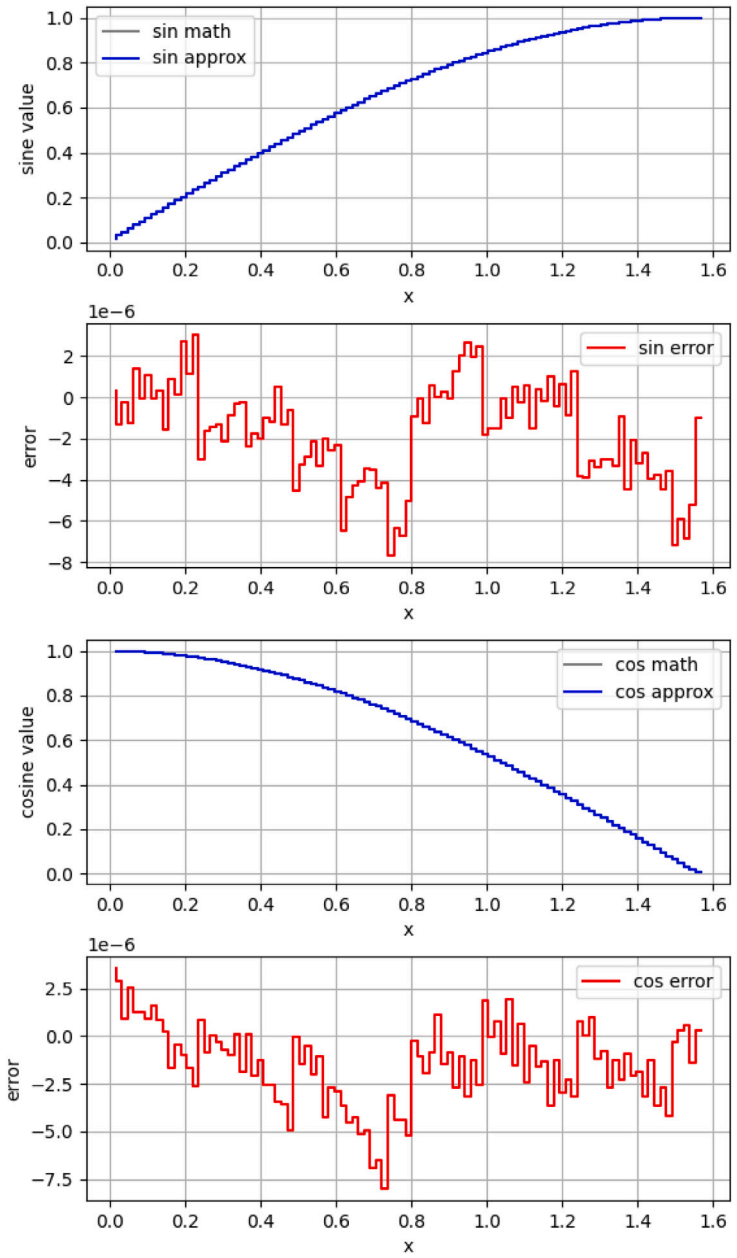


Fig. 9. Graphs of output values for the sin() and cos() functions (sin_math and cos_math – calculated function values in full precision, sin_approx and cos_approx – values from the presented algorithm; both graphs completely overlapped) and corresponding errors.

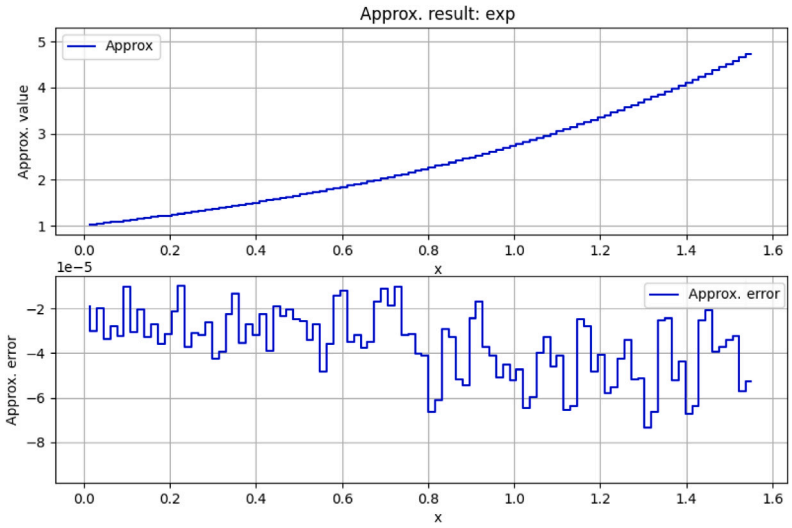


Fig. 10. Graphs of output value for the exp() function (Approx - values from the presented algorithm) and corresponding errors.

References

[1] J.E. Volder, The CORDIC trigonometric computing technique, IRE Trans. Electron. Comput. 8 (1959) 330–334, URL https://api.semanticscholar.org/CorpusID:206673426.

[2] J.S. Walther, A unified algorithm for elementary functions, in: Proceedings of the May 18-20, 1971, Spring Joint Computer Conference, in: AFIPS '71 (Spring), Association for Computing Machinery, New York, NY, USA, 1971, pp. 379–385, http://dx.doi.org/10.1145/1478786.1478840.

[3] G. Hekstra, E. Deprettere, Floating point cordic, in: Proceedings of IEEE 11th Symposium on Computer Arithmetic, 1993, pp. 130–137, http://dx.doi.org/10.1109/ARITH.1993.378100.

[4] S. Wang, V. Piuri, E. Wartzlander, Hybrid CORDIC algorithms, IEEE Trans. Comput. 46 (11) (1997) 1202–1207, http://dx.doi.org/10.1109/12.644295.

[5] D. Timmermann, H. Hahn, B. Hosticka, Low latency time CORDIC algorithms, IEEE Trans. Comput. 41 (8) (1992) 1010–1015, http://dx.doi.org/10.1109/12.156543.

[6] D. Timmermann, S. Dolling, Unfolded redundant CORDIC VLSI architectures With Reduced Area and power consumption, in: R. Reis, L. Claesen (Eds.), VLSI: Integrated Systems on Silicon: IFIP TC10 WG10.5 International Conference on Very Large Scale Integration 26–30 August 1997, Gramado, RS, Brazil, Springer US, Boston, MA, 1997, pp. 251–262, http://dx.doi.org/10.1007/978-0-387-35311-1_21.

[7] A. Madisetti, A. Kwentus, A. Willson, A 100-MHz, 16-b, direct digital frequency synthesizer with a 100-dBc spurious-free dynamic range, IEEE J. Solid-State Circuits 34 (8) (1999) 1034–1043, http://dx.doi.org/10.1109/4.777100.

[8] M. Kuhlmann, K.K. Parhi, P-CORDIC: A precomputation based rotation CORDIC algorithm, EURASIP J. Appl. Signal Process. 2002 (1) (2002) 936–943, http://dx.doi.org/10.1155/S1110865702205028.

[9] T.-B. Juang, S.-F. Hsiao, M.-Y. Tsai, Para-CORDIC: parallel CORDIC rotation algorithm, IEEE Trans. Circuits Syst. I. Regul. Pap. 51 (8) (2004) 1515–1524, http://dx.doi.org/10.1109/TCSI.2004.832734.

[10] T.-B. Juang, Low latency angle recoding methods for the higher bit-width parallel CORDIC rotator implementations, IEEE Trans. Circuits Syst. II: Express Briefs 55 (11) (2008) 1139–1143, http://dx.doi.org/10.1109/TCSII.2008.2002566.

[11] Y. Hu, S. Naganathan, An angle recoding method for CORDIC algorithm implementation, IEEE Trans. Comput. 42 (1) (1993) 99–102, http://dx.doi.org/10.1109/12.192217.

[12] M. Qin, T. Liu, B. Hou, Y. Gao, Y. Yao, H. Sun, A low-latency RDP-CORDIC algorithm for real-time signal processing of edge computing devices in smart grid cyber-physical systems, Sensors 22 (19) (2022) http://dx.doi.org/10.3390/s22197489.

[13] F. Curticapean, J. Niittylahti, A hardware efficient direct digital frequency synthesizer, in: ICECS 2001. 8th IEEE International Conference on Electronics, Circuits and Systems (Cat. No.01EX483), vol. 1, 2001, pp. 51–54, http://dx.doi.org/10.1109/ICECS.2001.957664.

[14] P. Poczekajlo, L. Moroz, E. Deelman, M. Taufer, P. Gepner, J. Krawiec, Low latency recoding CORDIC algorithm for fpga implementation, in: M.H. Lees, W. Cai, S.A. Cheong, Y. Su, D. Abramson, J.J. Dongarra, P.M.A. Sloot (Eds.), Computational Science – ICCS 2025, Springer Nature Switzerland, Cham, 2025, pp. 75–89, http://dx.doi.org/10.1007/978-3-031-97632-2_6.

[15] M.D. Ercegovac, T. Lang, CHAPTER 11 - CORDIC algorithm and implementations, in: M.D. Ercegovac, T. Lang (Eds.), Digital Arithmetic, in: The Morgan Kaufmann Series in Computer Architecture and Design, Morgan Kaufmann, San Francisco, 2004, pp. 608–648, http://dx.doi.org/10.1016/B978-155860798-9/50013-0.

[16] R. Andraka, A survey of CORDIC algorithms for FPGA based computers, in: Proceedings of the 1998 ACM/SIGDA Sixth International Symposium on Field Programmable Gate Arrays, FPGA '98, Association for Computing Machinery, New York, NY, USA, 1998, pp. 191–200, http://dx.doi.org/10.1145/275107.275139.

[17] P. Gepner, D.L. Fraser, M.F. Kowalik, Second generation quad-core intel xeon processors bring 45 nm technology and a new level of performance to HPC applications, in: Computational Science – ICCS 2008, ICCS 2008, in: Lecture Notes in Computer Science, vol. 5101, Springer, 2008, http://dx.doi.org/10.1007/978-3-540-69384-0_47.

[18] M. Ciznicki, P. Kopta, M. Kulczewski, K. Kurowski, P. Gepner, Elliptic solver performance evaluation on modern hardware architectures, in: Parallel Processing and Applied Mathematics, in: Lecture Notes in Computer Science, vol. 8384, Springer, 2014, http://dx.doi.org/10.1007/978-3-642-55224-3_16.

[19] P. Kopta, M. Kulczewski, K. Kurowski, T. Piontek, P. Gepner, M. Puchalski, J. Komasa, Parallel application benchmarks and performance evaluation of the intel xeon 7500 family processors, Procedia Comput. Sci. 4 (2011) 372–381, http://dx.doi.org/10.1016/j.procs.2011.04.039.

[20] P. Gepner, Using AVX2 instruction set to increase performance of high performance computing code, Comput. Inform. 36 (5) (2017) 1001–1018, URL https://www.cai.sk/ojs/index.php/cai/article/view/2017_5_1001.

[21] P. Gepner, V. Gamayunov, D.L. Fraser, Effective implementation of DGEMM on modern multicore CPU, Procedia Comput. Sci. 9 (2012) 126–135, http://dx.doi.org/10.1016/j.procs.2012.04.014.

[22] Scilab, 2025, URL https://www.scilab.org/. (Accessed 18 December 2025).